

Geotagged Pictures Map Application

yesmine.djebbi

May 2026

1 Technology Choices

1.1 Frontend

- Next.js (React + TypeScript)
- TailwindCSS
- React-Leaflet

Next.js provides a clean fullstack development experience with API routes, routing, and optimized frontend rendering. TypeScript improves maintainability and developer experience. TailwindCSS accelerates UI development. React-Leaflet allows simple integration of interactive maps using OpenStreetMap.

1.2 Backend

- Next.js API Routes
- Prisma ORM

Using Next.js API routes keeps frontend and backend inside the same repository, reducing setup complexity and improving developer productivity. Prisma provides type-safe database access and simplifies schema management.

1.3 Database

- PostgreSQL (Neon)

PostgreSQL is reliable, scalable, and well suited for relational data such as users, photos, and comments. Neon provides a serverless PostgreSQL solution with easy cloud deployment and automatic scaling.

1.4 Authentication

- JWT Authentication

JWT provides a lightweight and scalable authentication mechanism suitable for REST APIs and protected routes.

1.5 Image Storage

- Cloudinary

Images should not be stored directly inside the database because it increases database size and reduces performance. Only image URLs and metadata are stored in the database.

1.6 Map Provider

- OpenStreetMap + Leaflet

Leaflet is lightweight, open-source, and easy to integrate. OpenStreetMap avoids external paid map services during MVP development.

1.7 AI Integration

- OpenAI Vision API

The AI service automatically generates image descriptions after upload, improving user experience and demonstrating AI integration capabilities.

2 System Architecture

The system would follow a client-server architecture.

- The frontend communicates with backend REST APIs.
- Uploaded images are stored in Cloudinary.
- Metadata (GPS coordinates, image URL, AI description, comments, users) are stored in PostgreSQL.
- Prisma acts as the ORM layer between the backend and the database.
- JWT tokens are used to protect authenticated routes such as image upload and comments.

3 Database Models

3.1 User

Field	Type
id	UUID
email	String
password	String
createdAt	Date

3.2 Photo

Field	Type
id	UUID
imageUrl	String
latitude	Float
longitude	Float
aiDescription	Text
userId	Relation
createdAt	Date

3.3 Comment

Field	Type
id	UUID
text	String
photoId	Relation
userId	Relation
createdAt	Date

4 Authentication Flow

1. User signs up using email and password
2. Password is hashed before storage
3. User logs in and receives a JWT token
4. Protected API routes verify authentication before allowing uploads or comments

5 Image Upload and Data Flow

5.1 Upload Flow

1. User uploads an image from the frontend
2. Frontend sends the image to the backend API
3. Backend extracts EXIF GPS metadata
4. Backend uploads the image to Cloudinary
5. AI generates an automatic description
6. Backend stores:
 - image URL
 - latitude

- longitude
- AI description
- user reference

7. Frontend refreshes map markers

5.2 Map Visualization Flow

1. Frontend requests photo data from `/api/photos`
2. Backend returns photo metadata and coordinates
3. React-Leaflet renders markers on the map
4. Clicking a marker opens a popup containing:
 - image preview
 - AI description
 - comments section

6 Deployment Strategy

6.1 Frontend + Backend

- Vercel

6.2 Database

- Neon PostgreSQL

6.3 Image Storage

- Cloudinary

7 Scalability and Performance Considerations

7.1 Image Storage Trade-offs

Storing Images in Database

Advantages

- Simpler architecture
- Easier backups

Disadvantages

- Large database size

- Slower queries
- Poor scalability

Storing Images in Cloud Storage (Chosen Solution)

Advantages

- Better scalability
- CDN acceleration
- Optimized image delivery
- Reduced database load

Disadvantages

- External dependency
- Additional cost at scale

8 Handling Large Datasets (10k+ Photos)

Displaying thousands of markers simultaneously can reduce frontend performance. Several optimizations are required:

8.1 Marker Clustering

Markers close to each other are grouped together to reduce rendering load.

8.2 Bounding Box Queries

The backend only returns markers visible inside the current map viewport.

8.3 Lazy Loading

Images inside popups are loaded only when needed.

8.4 Database Indexing

Indexes on latitude and longitude improve geolocation query performance.

8.5 CDN Optimization

Cloudinary serves compressed and resized images to reduce bandwidth usage.

9 Estimated Development Timeline

Task	Estimated Time
Project setup & architecture	0.5 day
Database schema & Prisma setup	0.5 day
Authentication system	1 day
Image upload & Cloudinary integration	1 day
GPS metadata extraction	0.5 day
Map integration & marker system	1 day
Comments system	0.5 day
AI integration	1 day
Frontend UI/UX polishing	1 day
Testing & debugging	1 day
Deployment & environment setup	0.5 day
Total Estimated Time	Approx. 8 days